CSS 262: Linux Administration

Mandatory Access Control (SELinux)

Lecture 10: Confining Processes with SELinux

📋 Today's Agenda

Part 1: DAC vs MAC

- Why standard permissions aren't enough
- Discretionary vs. Mandatory Access Control
- Real-world breaches that MAC prevents

Part 2: SELinux Architecture

- Modes: enforcing, permissive, disabled
- Policies: targeted vs. MLS
- Checking & changing modes

Part 3: Security Contexts & Labels

- Users, roles, types, levels
- `ls -Z`, `ps -Z`, `id -Z`
- `chcon`, `restorecon`, `semanage fcontext`

Part 4: Booleans & Troubleshooting

- Tuning policy with `setsebool`
- `audit2why`, `audit2allow`
- Common SELinux denials & fixes

🔄 Quick Recap: Week 9

Firewalls & Packet Filtering

- Netfilter/iptables: chains, tables, targets
- Stateful inspection with conntrack
- UFW as iptables frontend
- Default deny, allow SSH first, persist rules

Key Takeaways

- Firewalls control **network** access (Layer 3/4)
- But what if an attacker gets **past** the firewall?
- A compromised Apache process can read `/etc/shadow`?
- **Today**: Confine processes to their own sandbox

🛡️ **Defense in Depth:** Firewalls guard the perimeter. SELinux guards what processes can do *after* they're running.

Part 1

🔓 DAC vs MAC

🔓 Discretionary Access Control (DAC)

How Traditional Linux Permissions Work

- Owner of a file **decides** who can access it
- `chmod`, `chown`, `chgrp` – user-controlled
- Root (UID 0) bypasses **all** permissions
- Processes inherit the permissions of the user who started them

The Problem

- If Apache runs as `www-data` and gets exploited…
- Attacker has **all** rights of `www-data`
- Can read anything `www-data` can read
- Root processes? **Unlimited access** if compromised
- DAC trusts the user – not the process

⚠️ **DAC Weakness:** A single compromised daemon can escalate to read sensitive files, open network connections, or pivot laterally – DAC can't stop it.

🛡️ Mandatory Access Control (MAC)

How MAC Works

- **System policy** decides access – not the user
- Even root is subject to MAC rules
- Each process is confined to a **security domain**
- Each file/resource has a **security label**
- Access = policy must explicitly allow domain → label

Benefits

- Compromised Apache can only access **Apache files**
- Root exploit in a service? Still confined by policy
- **Least privilege** enforced by the kernel
- Defense in depth: DAC + MAC + firewall
- Required by PCI-DSS, DISA STIG, HIPAA

✅ **MAC Principle:** "Even if you are root, if the policy says no, the answer is no."

📊 DAC vs MAC Comparison

Feature	DAC (Traditional)	MAC (SELinux)
Who sets policy	File owner	System administrator
Root bypass	Yes – root can do anything	No – root is confined too
Granularity	User/group/other	Process type → resource label
Default	Permissive (allow unless denied)	Restrictive (deny unless allowed)
Use case	Desktop, dev environments	Servers, compliance, production
Linux implementations	chmod, chown, ACLs	SELinux, AppArmor

💡 **DAC + MAC together:** Both must allow access. DAC says yes + MAC says no = **denied**.

🔑 **SELinux** is the MAC implementation on RHEL, CentOS, Fedora, and Amazon Linux. **AppArmor** is used on Ubuntu/Debian.

💥 Real-World: Why MAC Matters

Scenario: Apache Exploit Without SELinux

1. Attacker finds RCE in a PHP application
2. Gains shell as `www-data` (Apache's user)
3. Reads `/etc/shadow` → offline password cracking
4. Reads database credentials in `/var/www/config.php`
5. Opens reverse shell to external C2 server
6. **Full compromise** from a single web vulnerability

Same Scenario WITH SELinux (enforcing)

1. Attacker gains shell as `www-data` – but process is type `httpd_t`
2. `httpd_t` cannot read `/etc/shadow` → **denied**
3. `httpd_t` cannot open arbitrary network ports → **denied**
4. `httpd_t` can only access files labeled `httpd_sys_content_t`
5. Attacker is **confined** – limited blast radius

✅ **SELinux turns a full server compromise into a contained web-tier incident.**

Part 2

⚙️ SELinux Architecture

⚙️ SELinux Modes

Three Modes

- **Enforcing** – Policy is enforced; denials are blocked and logged
- **Permissive** – Policy is not enforced; denials are logged only
- **Disabled** – SELinux is completely off; no labels, no logging

Checking & Changing

```
# Check current mode
getenforce
# Output: Enforcing | Permissive | Disabled

# Temporarily switch (until reboot)
sudo setenforce 0   # Permissive
sudo setenforce 1   # Enforcing

# Check detailed status
sestatus
```

✅ **Production:** Always `Enforcing`. Use `Permissive` only for debugging.

🚨 **Never disable SELinux!** "Just disable SELinux" is the #1 bad advice on the internet. Fix the policy instead.

📄 Persistent Mode Configuration

`/etc/selinux/config`

```
# /etc/selinux/config
SELINUX=enforcing
SELINUXTYPE=targeted
```

Setting	Options	Description
`SELINUX`	`enforcing`, `permissive`, `disabled`	Mode after reboot
`SELINUXTYPE`	`targeted`, `mls`	Policy type

⚠️ **Changing to/from** `disabled` **requires a reboot** and full filesystem relabel (`touch /.autorelabel && reboot`). This can take 10+ minutes.

💡 **Targeted policy:** Confines specific services (httpd, sshd, etc.) while leaving unconfined processes alone. This is the default and most practical.

🏗️ SELinux Policy Architecture

How SELinux Decides

```
Process (subject)        Resource (object)
   httpd_t        →         httpd_sys_content_t
       ↓                           ↓
   "Can httpd_t read files labeled httpd_sys_content_t?"
       ↓
   Policy says: ALLOW ✅  (or DENY ❌)
```

Policy Components

Component	Description
Type Enforcement (TE)	Core – which types can access which types
Role-Based Access Control (RBAC)	Which roles can transition to which types
Multi-Level Security (MLS)	Sensitivity levels (classified, secret, top-secret)

💡 **Type Enforcement** is what you'll work with 99% of the time. Types on processes ("domains") and types on files ("labels") are the core concept.

📊 sestatus – Full Status Check

```
$ sestatus
SELinux status:                 enabled
SELinuxfs mount:                /sys/fs/selinux
SELinux root directory:         /etc/selinux
Loaded policy name:             targeted
Current mode:                   enforcing
Mode from config file:          enforcing
Policy MLS status:              enabled
Policy deny_unknown status:     allowed
Memory protection checking:     actual (secure)
Max kernel policy version:      33
```

💡 **Key fields:** "Current mode" = runtime mode. "Mode from config file" = mode after reboot. These can differ if you used `setenforce`.

✅ **Loaded policy:** `targeted` means only specific services are confined. Unconfined processes run as `unconfined_t`.

Part 3

Security Contexts & Labels

🏷️ SELinux Security Contexts

Every Process and File Has a Context

```
user:role:type:level
```

Field	Meaning	Example
user	SELinux user (not Linux user)	`system_u`, `unconfined_u`
role	Role for RBAC	`system_r`, `object_r`
type	Domain (process) or label (file)	`httpd_t`, `httpd_sys_content_t`
level	MLS sensitivity	`s0`, `s0:c0.c1023`

🔑 **The type field is what matters most.** Type Enforcement rules use it to decide access: "Can domain `httpd_t` read type `httpd_sys_content_t`?"

💡 Convention: process types end in `_t`, file types end in `_t`, booleans often end in `_bool`.

🔍 Viewing Contexts: `-Z` Flag

Files

```
$ ls -Z /var/www/html/index.html
unconfined_u:object_r:httpd_sys_content_t:s0  /var/www/html/index.html
```

Processes

```
$ ps auxZ | grep httpd
system_u:system_r:httpd_t:s0    root  1234  ... /usr/sbin/httpd
```

Current User

```
$ id -Z
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

Ports

```
$ sudo semanage port -l | grep http
http_port_t    tcp    80, 443, 8080, 8443
```

✅ **Remember:** Add `-Z` to `ls`, `ps`, `id`, `netstat` to see SELinux contexts.

🔧 Changing File Contexts: `chcon`

Temporary Context Change

```
# Change type of a file
sudo chcon -t httpd_sys_content_t /var/www/myapp/index.html

# Change full context
sudo chcon system_u:object_r:httpd_sys_content_t:s0 /var/www/myapp/index.html

# Recursive
sudo chcon -R -t httpd_sys_content_t /var/www/myapp/
```

⚠️ `chcon` **is temporary!** A filesystem relabel (`restorecon`) will overwrite your changes. Use `semanage fcontext` for permanent changes.

💡 **Use** `chcon` **for quick testing**, then make it permanent with `semanage fcontext` + `restorecon` .

🔄 Restoring Contexts: `restorecon`

Reset to Default Labels

```
# Restore default context for a file
sudo restorecon -v /var/www/html/index.html

# Recursive — restore entire directory
sudo restorecon -Rv /var/www/

# Check what would change (dry run)
sudo restorecon -Rvn /var/www/
```

When to Use

- After moving/copying files (context may not match destination)
- After `chcon` experiments – reset to known good
- After adding rules with `semanage fcontext`
- Full system relabel: `touch /.autorelabel && reboot`

💡 `cp` **inherits destination context;** `mv` **preserves source context.** This is the #1 cause of SELinux issues with moved files!

📝 Permanent Context Rules: `semanage fcontext`

Add a Permanent File Context Rule

```
# Add rule: all files under /srv/myapp get httpd_sys_content_t
sudo semanage fcontext -a -t httpd_sys_content_t "/srv/myapp(/.*)?"

# Apply the rule
sudo restorecon -Rv /srv/myapp/
```

Manage Rules

```
# List all custom rules
sudo semanage fcontext -l -C

# Delete a rule
sudo semanage fcontext -d "/srv/myapp(/.*)?"
```

✅ **Workflow:** 1) `semanage fcontext -a` → 2) `restorecon -Rv` → context survives relabels

💡 The regex `(/.*)?` matches the directory and everything inside recursively

🌐 Managing Port Contexts

SELinux also controls which ports services can bind to

```
# List allowed ports for HTTP
sudo semanage port -l | grep http_port_t
# http_port_t    tcp    80, 443, 8080, 8443

# Add a custom port (e.g., run Apache on 8888)
sudo semanage port -a -t http_port_t -p tcp 8888

# Delete a port mapping
sudo semanage port -d -t http_port_t -p tcp 8888
```

⚠️ **Common error:** Starting Apache on port 9090 fails with "Permission denied" even as root – SELinux blocks it because 9090 isn't in `http_port_t`.

✅ **Fix:** `sudo semanage port -a -t http_port_t -p tcp 9090` – then restart the service.

Part 4

🔧 Booleans & Troubleshooting

🔘 SELinux Booleans

What Are Booleans?

- **On/off switches** for policy features
- Toggle without writing custom policy
- Example: "Allow httpd to connect to the network"
- Hundreds of booleans available

Common Booleans

```
1   httpd_can_network_connect     off
2   httpd_can_sendmail            off
3   httpd_enable_homedirs         off
4   samba_enable_home_dirs        off
5   ftpd_full_access              off
```

Managing Booleans

```
1   # List all booleans
2   getsebool -a
3
4   # Filter
5   getsebool -a | grep httpd
6
7   # Set temporarily (until reboot)
8   sudo setsebool httpd_can_network_connect on
9
10  # Set permanently
11  sudo setsebool -P httpd_can_network_connect on
12
13  # List with descriptions
14  sudo semanage boolean -l | grep httpd
```

🔘 Common Boolean Scenarios

Scenario	Boolean	Default
Apache connects to DB backend	`httpd_can_network_connect_db`	off
Apache connects to any network	`httpd_can_network_connect`	off
Apache sends email	`httpd_can_sendmail`	off
Apache serves user home dirs	`httpd_enable_homedirs`	off
Samba shares home directories	`samba_enable_home_dirs`	off
FTP allows uploads	`ftpd_anon_write`	off
HTTPD can execute CGI	`httpd_enable_cgi`	on

💡 **Booleans are the easiest fix.** Before writing custom policy, check if a boolean already exists for your use case.

⚠️ **Use** `-P` **flag** to persist across reboots! Without it, boolean resets on next boot.

🔍 Troubleshooting: Reading AVC Denials

Where Denials Are Logged

```
# Audit log (primary)
sudo ausearch -m AVC -ts recent

# System journal
sudo journalctl -t setroubleshoot

# Messages log
sudo grep "avc:  denied" /var/log/audit/audit.log
```

Anatomy of an AVC Denial

```
type=AVC msg=audit(1234567890.123:456): avc:  denied  { read }
  for pid=1234 comm="httpd" name="config.php"
  dev="sda1" ino=789012
  scontext=system_u:system_r:httpd_t:s0
  tcontext=unconfined_u:object_r:user_home_t:s0
  tclass=file permissive=0
```

🔑 **Key fields:** `scontext` = source (process), `tcontext` = target (file), `{ read }` = denied action, `tclass` = object class.

🛠️ `audit2why` – Explain the Denial

Translate AVC Messages into Human-Readable Explanations

```
# Pipe audit log through audit2why
sudo ausearch -m AVC -ts recent | audit2why
```

Example Output

```
type=AVC msg=audit( ... ): avc:  denied  { read } for  ...
  scontext=system_u:system_r:httpd_t:s0
  tcontext=unconfined_u:object_r:user_home_t:s0

    Was caused by:
        Missing type enforcement (TE) allow rule.
        You can use audit2allow to generate a loadable module to allow this access.

    # Or: "The boolean httpd_enable_homedirs is set to off"
    #     setsebool -P httpd_enable_homedirs on
```

✅ `audit2why` **often tells you the exact boolean to flip.** Always check this before writing custom policy.

🛠️ `audit2allow` – Generate Custom Policy

When No Boolean Exists

```
# Generate a TE rule
sudo ausearch -m AVC -ts recent | audit2allow

# Generate a loadable policy module
sudo ausearch -m AVC -ts recent | audit2allow -M mypolicy
sudo semodule -i mypolicy.pp
```

Workflow

```
1. Hit denial in enforcing mode
2. Check audit.log → ausearch -m AVC
3. Run audit2why → is there a boolean?
   ├─ Yes → setsebool -P <boolean> on
   └─ No  → audit2allow → review → semodule -i
4. Test again
```

🚨 **Never blindly run** `audit2allow` **and install!** Review the generated rules. Overly permissive custom policy defeats the purpose of SELinux.

🔄 SELinux Troubleshooting Workflow

```
Service not working?
  ↓
Check: getenforce → is SELinux enforcing?
  ↓ Yes
Check logs: ausearch -m AVC -ts recent
  ↓
Run: audit2why
  ↓
├─ "Boolean X is off"     → setsebool -P X on
├─ "Wrong file context"   → restorecon -Rv /path
├─ "Wrong port context"   → semanage port -a -t type -p tcp PORT
└─ "Missing TE rule"      → audit2allow -M policy → review → semodule -i
  ↓
Test the fix
  ↓
Verify: ausearch -m AVC -ts recent (no new denials)
```

💡 **Tip:** Switch to permissive (`setenforce 0`), reproduce the issue, collect all denials, fix them, then go back to enforcing (`setenforce 1`).

📝 `setroubleshoot` – GUI-Friendly Diagnostics

Install & Use

```
# Install setroubleshoot
sudo yum install setroubleshoot setroubleshoot-server

# View recent alerts
sudo sealert -a /var/log/audit/audit.log

# Real-time monitoring
sudo journalctl -t setroubleshoot -f
```

Example Output

```
SELinux is preventing httpd from read access on the file config.php

***** Plugin restorecon (99.5 confidence) suggests *****
If you want to fix the label:
    /sbin/restorecon -v /var/www/html/config.php
```

✅ `sealert` **ranks solutions by confidence.** Start with the highest-confidence suggestion.

⚠️ Common SELinux Pitfalls

Pitfall	Cause	Fix
File copied with `mv`	`mv` preserves source context	`restorecon -Rv /destination`
Custom web root	Files not labeled `httpd_sys_content_t`	`semanage fcontext` + `restorecon`
Non-standard port	Port not in SELinux port list	`semanage port -a`
App can't connect to DB	`httpd_can_network_connect_db` is off	`setsebool -P`
Home dir in web root	`httpd_enable_homedirs` is off	`setsebool -P` or fix file labels
"Just disable SELinux"	Nukes all MAC protection	Fix the actual denial instead

🚨 **#1 Rule:** Never disable SELinux. If it's blocking something, the fix is almost always: wrong label, missing boolean, or missing port context.

✅ SELinux Security Checklist

Configuration

- ☐ SELinux is in **enforcing** mode
- ☐ `/etc/selinux/config` says `enforcing`
- ☐ Targeted policy loaded
- ☐ No unnecessary custom modules
- ☐ File contexts match expected labels
- ☐ Non-standard ports registered with `semanage`

Operations

- ☐ Monitor denials: `ausearch -m AVC`
- ☐ Use `audit2why` before custom policy
- ☐ Use `restorecon` after moving files
- ☐ Set booleans with `-P` flag (persist)
- ☐ Document any custom policy modules
- ☐ Test changes in permissive, deploy in enforcing

📊 Summary: Mandatory Access Control (SELinux)

Key Concepts

1. **DAC vs MAC**: DAC = user decides; MAC = policy decides
2. **Modes**: Enforcing, Permissive, Disabled
3. **Contexts**: `user:role:type:level` on every object
4. **Type Enforcement**: Core — `domain_t` can access `type_t`
5. **Booleans**: Toggle policy features on/off
6. **Troubleshooting**: `ausearch` → `audit2why` → fix

Essential Commands

Command	Purpose
`getenforce` / `sestatus`	Check mode & status
`ls -Z` / `ps -Z`	View contexts
`chcon -t TYPE`	Temporary label change
`restorecon -Rv`	Restore default labels
`semanage fcontext`	Permanent context rules
`setsebool -P`	Toggle booleans
`ausearch -m AVC`	Find denials
`audit2why`	Explain denials

🎯 Learning Objectives: Did We Achieve?

By now, you should be able to:

- ✅ Explain the difference between DAC and MAC
- ✅ Describe SELinux modes and when to use each
- ✅ Read and interpret security contexts (`user:role:type:level`)
- ✅ Use `ls -Z` , `ps -Z` , `id -Z` to inspect contexts
- ✅ Change file contexts with `chcon` and `restorecon`
- ✅ Create permanent context rules with `semanage fcontext`
- ✅ Manage SELinux booleans with `getsebool` and `setsebool -P`
- ✅ Troubleshoot denials using `ausearch` , `audit2why` , and `sealert`

🎓 **Next Week:** Logging, Auditing & Cron – journald, rsyslog, logrotate, auditd, and scheduled tasks!

🧪 Lab Practice: SELinux Contexts

Exercise 1: Check & Toggle Modes

Verify SELinux is enforcing with `getenforce` and `sestatus`. Switch to permissive with `setenforce 0`, confirm, then switch back. Check `/etc/selinux/config` for boot-time setting.

Exercise 2: File Contexts

Create a file in `/tmp/`, then move it to `/var/www/html/`. Check its context with `ls -Z`. Use `restorecon` to fix it. Verify the context changed to `httpd_sys_content_t`.

Exercise 3: Custom Web Root

Create `/srv/mysite/index.html`. Use `semanage fcontext` to assign `httpd_sys_content_t`. Apply with `restorecon`. Configure Apache to serve from `/srv/mysite` and verify it works.

Exercise 4: Booleans & Troubleshooting

Attempt to have Apache connect to a backend service. Check `audit.log` for the AVC denial. Run `audit2why` to identify the boolean. Enable it with `setsebool -P` and verify the connection works.

🔗 Additional Resources

Documentation

- Red Hat SELinux User's Guide – Comprehensive reference
- SELinux Project Wiki – Community documentation
- Fedora SELinux Guide – Getting started

Books & Guides

- *"SELinux System Administration"* by Sven Vermeulen – Deep dive into SELinux
- Gentoo SELinux Handbook – Thorough technical reference
- CIS Benchmark – SELinux hardening section

Practice

- Install RHEL/CentOS VM with SELinux enforcing
- Deploy Apache with a custom web root and fix all SELinux denials
- Use `audit2why` to troubleshoot Samba or PostgreSQL access issues

Questions?

Next: Logging, Auditing & Cron